

EXAMEN TRANSVERSAL

DESARROLLO FULLSTACK III

Informe Retrospectivo y Defensa Técnica

Estudiantes : Vicente Sánchez – Felipe Cáceres Asignatura Desarrollo FullStack III – 001D

Docente Evaluador : Rodrigo Chacón Sede & Código : Sede Alameda – Código: DSY1106

| Tabla de Contenidos

01 Arquitectura de Microservicios

02 Decisiones del Stack Tecnológico

03 Aplicación de Patrones de Diseño

04 Estrategia de Branching (GitFlow)

05 Integración y Despliegue (Docker)

06 Calidad y Pruebas Unitarias

07 Ética, Privacidad y Sostenibilidad

08 Retrospectiva y Conclusiones

| DISEÑO DE ARQUITECTURA

Microservicios y DDD

Se implementó una arquitectura distribuida para garantizar el desacoplamiento. Cada servicio responde a un dominio específico: Auth, Products y Orders.

- Escalabilidad: Crecimiento independiente por servicio.
- Comunicación: Intercambio vía API REST JSON.
- Resiliencia: Aislamiento de fallos por componente.

Justificación

Esta arquitectura permite que un error en el módulo de ventas no comprometa la disponibilidad del catálogo, asegurando la continuidad operativa del negocio.

DECISIONES DE DESARROLLO

Backend: Node.js

Uso de I/O no bloqueante. Ideal para microservicios con alta concurrencia de peticiones ligeras.

Frontend: React.js

Basado en componentes reutilizables. El Virtual DOM garantiza actualizaciones de UI eficientes.

Mantenibilidad

Estructura modular separando rutas y controladores, facilitando la escalabilidad del sistema.

Eficiencia

Middlewares para validaciones y gestión de errores, eliminando la redundancia en el código.

| APLICACIÓN DE PATRONES

Modularidad

Los patrones permiten que la aplicación sea adaptable a cambios en la infraestructura sin afectar la lógica.

Repository Pattern: Desacopla la lógica de negocio del acceso a datos (PostgreSQL/MongoDB).

Optimización

- DTO: Envío de campos estrictamente requeridos.
- Middleware: Seguridad y logs uniformes.
- Singleton: Gestión única de conexiones a DB.

| ESTRATEGIA DE BRANCHING

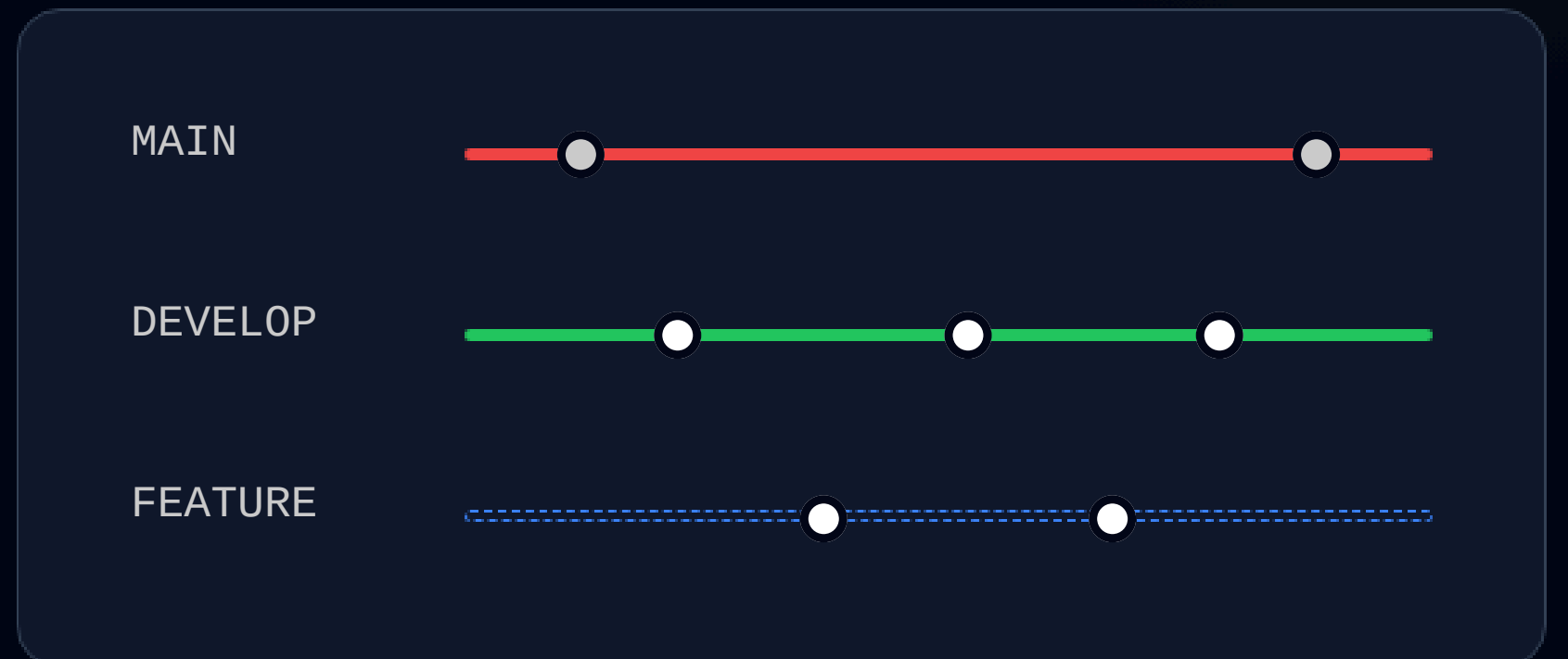
GitFlow Workflow

Estrategia para organizar el flujo grupal, asegurando la integridad de la rama de producción.

● Main: Versiones productivas.

● Develop: Integración del equipo.

● Features: Desarrollo de funciones.



ASEGURAMIENTO DE CALIDAD

85%

COBERTURA JEST

Validación con Tests

Pruebas unitarias para validar la lógica de negocio de forma automática.

- Estabilidad: Prevención de errores antiguos.
- Confianza: Refactorización segura del código.
- Calidad: Documentación del comportamiento esperado.

| ÉTICA Y SOSTENIBILIDAD

Privacidad

Encriptación BCrypt y uso de variables de entorno para proteger datos sensibles.

Responsabilidad

Seguridad OWASP para prevenir ataques de SQL Inyección y Cross-Site Scripting (XSS).

Sostenibilidad

Consultas optimizadas para reducir el consumo de recursos y la latencia de red.

Escalabilidad

Diseño modular que permite crecer sin comprometer la integridad de la información.

| ANÁLISIS RETROSPECTIVO

Aciertos y Logros

La arquitectura de microservicios permitió independencia en el desarrollo y un despliegue ágil mediante contenedores Docker.

Oportunidades

A futuro, se propone implementar un Service Mesh para gestionar la telemetría y seguridad avanzada entre microservicios.

"Software resiliente bajo estándares modernos de ingeniería."

¿PREGUNTAS O DUDAS?

Defensa Oral – DSY1106 | Duoc UC

